

UNIT 1: BIG DATA

SECTION 1: BIG DATA DEFINITION

1.1 Definition

There is no single standard definition of Big Data. The working definition used in this course:

Big Data is data whose scale, diversity, and complexity require new forms of processing, using new architectures, techniques, algorithms, and analytics, to manage the data and enhance decision making by extracting value and hidden knowledge or insights from it.

1.2 How Big Data Differs from Traditional Data Processing

Aspect	Traditional Data Processing	Big Data Processing
Schema	Fixed, defined in advance	Flexible, often schema-on-read
Data quality	Clean and consistent	Messy, inconsistent
Processing style	Batch, scheduled (e.g. once a day)	Batch and real-time both possible
Storage	Centralized (single server, RDBMS)	Distributed (across many machines)
Data generators	Enterprises only	Everybody (users, sensors, devices)
Data types	Structured only	Structured, semi-structured, unstructured
Model creation	Human generated, based on domain rules	Machine generated, based on statistical patterns in data

1.3 Big Data and Analytics Pipeline

Input Data -> Model -> Analysis

Model: a human construct that helps understand real-world systems or phenomena.

- Represents the inherent properties of the data.
- Can represent a map or schema, grouping data elements such as records or tables and their associations.
- Does not depend on the software using that data.

1.4 Two Foundational Themes in Big Data

1. Data Management: the infrastructure and processes needed to store, process, and govern massive datasets. This is the foundation that makes analytics possible.
2. Data Analytics: once data is properly managed, analytical techniques are applied to extract value and knowledge, uncovering insight that drives decisions.

1.5 Real World Examples

Google Search System: uses automated web crawlers that find and index billions of pages. When a user queries, the system analyzes many factors, including words, content, and source expertise, to return the most relevant and reliable results.

Amazon Recommendation System: uses machine learning algorithms and collaborative filtering to analyze vast amounts of user data such as purchase history, browsing behavior, and product ratings.

1.6 Traditional vs Big Data Approach: Machine Translation Example

Task: translate a sentence from English to Hindi.

Traditional Approach (rule based, linguistic):

- Requires a linguistic expert to build the model.
- Model must include language structure (morphology, grammar), meaning of words, and word to word mapping between languages.
- The model is human generated.

Big Data Approach (statistical):

- Makes no attempt to understand language at all.
- Gathers a large parallel corpus, meaning millions of sentence pairs with their translations.
- Builds a statistical model from patterns. Example: every time “cat” appears in the English sentence and “billi” appears in the paired Hindi sentence, infer that cat translates to billi.
- The model is machine generated.

Key quote: “All models are wrong, and increasingly you can succeed without them.” Peter Norvig, Google Research Director, in the paper “The Unreasonable Effectiveness of Data.”

CHECK YOUR UNDERSTANDING (Section 1)

1. Define Big Data in your own words and explain how it differs from traditional data processing.
(Question Bank Q1)
2. What does it mean for a model to be “machine generated” rather than “human generated”? Use the machine translation example to explain.
3. Why is data management called the “foundation” that makes data analytics possible?
4. Explain Amazon’s use of collaborative filtering in one or two sentences.

SECTION 2: THE FIVE Vs OF BIG DATA

Big Data is commonly described using characteristics known as the Vs. The core four covered in this course are Volume, Variety, Velocity, and Veracity. A fifth, Value, is commonly added in industry usage.

2.1 Volume (Scale)

Old Style Data:

- Fixed schema and format, ensuring consistency and structure.
- Data is clean and consistent, making it reliable for analysis.
- Data rates are predictable, simplifying planning.
- Uses an elaborate ETL (Extract, Transform, Load) procedure, run in batches, perhaps once a day.
- Output is primarily reports, generated by batch queries.
- Generated only by enterprises.

Big Data:

- Data volume increasing exponentially. A 44x increase from 2009 to 2020 (from 0.8 zettabytes to 35 zettabytes).
- Generated by everybody, not just enterprises.
- Collected automatically:
 - Financial services: every stock trade, bank transaction recorded (India: no paper shares anymore).
 - Energy and utilities: sensors placed in every home (smart grid).
 - Communications, media, and internet services generate continuous data.

Real world example: The banking sector's shift from manual ledger entries to fully automated, every-transaction-recorded digital systems is a Volume example.

2.2 Variety

Why analyzing Big Data is complex:

- Data comes in many formats and structures: text, numerical, images, audio, video, sequences, time series, social media data, multi-dimensional arrays.

- Static data versus streaming data.
- A single application can generate or collect many types of data at once.

Old Style Data: fixed format and schema.

Big Data: must integrate varied sources such as Twitter, Maps, Facebook. To extract knowledge, all these different types of data must be linked together.

Real world example: The Chennai Floods crisis mapping project combined human input and machine learning on social media data (a mix of text, images, and geotags) to track disaster impact in real time.

2.3 Velocity

Old Style Data: input data collected at night, output as a daily report (batch, delayed).

Big Data: real time input, real time response.

Real world example: Nokia's real time mobile network analytics correlates network wide data instantly with customer experience, allowing immediate fault detection.

2.4 Veracity

Veracity refers to the messiness or trustworthiness of data, which affects its quality and reliability.

- Accuracy is difficult to control due to hashtags, typos, and abbreviations common in unstructured data sources.
- Analysts must work with such messy data despite this, because it often contains valuable insight.

Old Style Data: typically manually entered into fixed fields, resulting in a low chance of error and high reliability.

Big Data: inconsistent data.

Real world example: In India, a person can end up with multiple PAN cards because their Aadhar and PAN details are not consistent, for example slight variations in name and address such as "PES University, 100 ft Road" versus "Pes University, Dwaraka Nagar." Names and addresses do not match exactly, yet postal delivery still works because humans interpret the inconsistency.

2.5 Value (fifth V, general industry usage)

Value refers to the ability to turn raw data into something useful and actionable. Collecting large amounts of data (Volume) with high Variety, Velocity, and Veracity concerns is only worthwhile if

the analysis produces business or scientific value, such as better decisions, new revenue, or new discoveries. Value is the end goal that justifies investment in handling the other four Vs.

CHECK YOUR UNDERSTANDING (Section 2)

1. Explain the five Vs of Big Data with real world examples for each. (Question Bank Q2)
2. Why is Veracity a particular challenge for social media data specifically?
3. Give an example, other than the ones in the notes, of a Volume driven data source generated by “everybody” rather than an enterprise.
4. How does Variety create difficulty when trying to extract knowledge from multiple data sources together?

SECTION 3: CHALLENGES IN MANAGING AND ANALYZING BIG DATA

3.1 Domain Knowledge Gaps

Even a purely statistical, data driven model can fail without domain knowledge.

Example: the idiom “let the cat out of the bag” (meaning: reveal a secret).

- A naive statistical translation produces a literal translation such as “billi ko bag ke bahar chhod diya,” which is wrong because it loses the idiomatic meaning.
- Fixing this requires domain and language knowledge combined with experimentation.

3.2 Spurious Correlation

If C causes A, and C causes B, this does NOT mean A causes B.

Classic example: Do storks deliver babies? A chart shows positive correlation between stork population and human birth rates across European countries. What it does not show is a hidden variable: available nesting area. Larger regions simply have more storks and more people.

3.3 Gaps in the Data: Selection Bias

Selection bias occurs when the sample data used is not representative of the larger population, leading to skewed or incorrect conclusions because the sample systematically excludes certain groups. This is often caused by convenience, meaning researchers use data sources that are easy to access.

Example: The Rutgers University study used tweets to examine emergency decision making during Hurricane Sandy. This was convenient, but most tweets came from a single location, Manhattan. The sample was not representative of all hurricane affected areas, so using this data would lead to a skewed conclusion that Manhattan was the most affected area.

Example (medicine): missing data is a constant challenge, and “negative results” (failed trials or no measurable effect) are more likely to go unreported. This produces a biased sample that overestimates the benefits of treatments.

3.4 Real Time Analysis Challenges

Real time Big Data applications face additional challenges beyond the above:

- High Velocity data streams must be processed and analyzed with very low latency, which strains traditional batch oriented systems.
- Data quality and Veracity issues are harder to catch and correct in real time compared to batch processing, where cleaning steps can be applied before analysis.
- Scalability: the volume of incoming data can spike unpredictably (for example during a viral event or a natural disaster), and the system must scale to handle bursts.
- Fault tolerance under time pressure: a failed node during real time processing cannot simply be retried later the way a batch job can, since the value of the result often depends on immediacy.
- Storage versus processing tradeoff: real time systems often must decide what to persist for later versus what to discard after immediate use, since storing everything at high velocity is expensive.

3.5 How These Challenges Are Addressed

Summary of methods:

1. Use domain knowledge to check the model for validity.
2. Estimate errors empirically.

Expert example, Nate Silver: author of “The Signal and the Noise,” named on Time Magazine’s 2009 list of 100 most influential people, correctly predicted the US 2008 and 2012 elections by combining statistics with human adjustment.

Weather forecasting example (why it succeeds despite being a chaotic system):

- Weather is a chaotic, dynamic, nonlinear system where tiny changes cause large divergence.
- Humans adjust the model by computing probabilities, such as how often “rain predicted” actually matched reality, and validating against ground truth.
- Commercial forecasting also factors in customer satisfaction, since people are more upset by an unpredicted “no rain” turning into rain than the reverse, so models are tuned to be cautious about that specific error type.

Empirical error estimation method:

1. Split data into a training set and a testing set.
2. Develop the algorithm on the training set.
3. Measure accuracy by estimating error on the unseen testing set.

This is the standard way to compare different analytics algorithms, and it is used in examples

such as Amazon's recommendation system, where a model built from historical data is evaluated by tracking how many people click or purchase the recommended items.

CHECK YOUR UNDERSTANDING (Section 3)

1. What are the key challenges in managing and analyzing Big Data in real time applications?
(Question Bank Q3)
2. Explain selection bias using the Hurricane Sandy Twitter example, and explain why it produced a misleading conclusion.
3. Why does the stork and birth rate example illustrate the danger of confusing correlation with causation?
4. Describe, step by step, how the empirical training and testing method is used to estimate error in a Big Data algorithm.

SECTION 4: OPPORTUNITIES WITH BIG DATA

4.1 Healthcare

- Predicting disease outbreaks by analyzing vast amounts of health records.
- Improving patient care through analysis of patient history and trends.
- Customized or personalized healthcare, tailoring treatment to individual differences in genetics, age, and gender.
- Genomics research (see Section 5) uses Big Data techniques to sequence and analyze DNA at scale.

4.2 Finance

- Automated, real time recording of every transaction (stock trading, banking) enables fraud detection and risk analysis at a scale impossible with manual processes.
- Statistical, data driven forecasting models (as in the Nate Silver election prediction example) can be applied to financial market prediction and risk modeling.
- Large scale analytics on structured transaction data combined with unstructured data (news, social sentiment) supports better informed trading and lending decisions.

4.3 Agriculture

- Sensor based data collection (soil sensors, weather stations, satellite imagery) enables precision farming, where irrigation, fertilization, and pest control decisions are made based on real, localized data rather than generalized assumptions.
- Predictive analytics can forecast crop yields and detect early signs of disease or drought stress across large farmland areas.
- Supply chain optimization for agricultural produce becomes possible when data from farm to market is tracked and analyzed at scale.

CHECK YOUR UNDERSTANDING (Section 4)

1. Discuss the opportunities Big Data brings in the fields of healthcare, finance, and agriculture. (Question Bank Q4)
2. How does the concept of “customized healthcare” connect to the Variety characteristic of Big Data discussed in Section 2?

3. Give one example each of a Volume driven, a Velocity driven, and a Veracity driven opportunity in the finance sector.

SECTION 5: DATA INTENSIVE SCIENTIFIC DISCOVERY

5.1 Concept

Big Data plays a growing role in scientific discovery, where insight is obtained from analyzing massive datasets rather than purely from hypothesis driven experiments. This is sometimes referred to as a new, data intensive approach to science, alongside the traditional empirical, theoretical, and computational approaches.

5.2 Genomics Example

Gene sequencing of DNA fragments found in the ocean, carried out by J. Craig Venter:

- Result: discovery of thousands of new species.
- Key characteristic: there was no prior idea of what these species look like or any other biological information. The discovery emerged purely from the data, meaning the sequence patterns, not from traditional observation first science.
- This illustrates the opportunity: Big Data enables discovery at a scale and speed impossible with traditional, manual, hypothesis first scientific methods.
- It also reinforces a challenge: data only discovery risks being correlation without understanding, so domain expertise is still required to interpret and validate findings.

5.3 Astronomy Example (general knowledge, standard course context)

Modern astronomical sky surveys generate enormous volumes of imaging and spectral data every night, far beyond what any team of astronomers could manually inspect.

- Automated pipelines process this data to detect transient events, such as supernovae, in near real time, flagging candidates for follow up observation.
- Statistical and machine learning techniques are used to classify celestial objects at scale, since manual classification of millions of objects is infeasible.
- This mirrors the genomics case: patterns are found in the data first, and scientific understanding is built around what the data reveals, rather than the reverse.

5.4 The Pipeline in a Scientific Context

Input Data -> Model -> Analysis

In data intensive science, the model can emerge from the data itself, meaning it is machine generated, rather than being hand built by domain scientists in advance. This is a genuine shift from traditional scientific modeling, where the model is constructed first and then tested against data.

CHECK YOUR UNDERSTANDING (Section 5)

1. Elaborate on the role of Big Data in data intensive scientific discovery, with examples from astronomy or genomics. (Question Bank Q5)
2. In the Venter genomics example, why is discovering thousands of new species without knowing what they look like considered both an opportunity and a limitation?
3. Explain, in your own words, how data intensive science differs from traditional hypothesis driven science.

SECTION 6: HISTORICAL EVOLUTION OF BIG DATA

6.1 Foundation

Big Data is a fusion of established statistics and the newer field of computer science, building on existing knowledge from mathematics and data analysis.

6.2 The Data Deluge

A significant shift occurred in the twenty first century, where the volume and speed of data generation exploded. The total amount of data in the world grew from 4.4 zettabytes in 2013 to a projected 44 zettabytes by 2020. This massive, unmanageable volume of data transformed traditional data analysis into what is now known as Big Data.

6.3 Three Major Phases in the Evolution of Big Data

Phase	Time Period	Key Milestone
Phase 1	1970s	Structured Content: driven by database management and technologies like SQL
Phase 2	Early 2000s	Web Based Unstructured Content: rise of the internet and web applications created vast unstructured data from search queries and user interactions
Phase 3	Current	Mobile and Sensor Based Content: proliferation of mobile devices and the Internet of Things (IoT), generating high volume, real time data every second

6.4 Key Technology Milestones (built into later units, listed here as a timeline)

- 2003: Google publishes the Google File System (GFS) paper.
- 2004: Google publishes the MapReduce paper (OSDI 2004).
- Mid to late 2000s: Yahoo develops Hadoop as an open source implementation of GFS and MapReduce concepts, later becoming an Apache project.
- 2010: MapReduce version 2 introduced with YARN, addressing the scalability limits of the original Hadoop 1.0 JobTracker design.
- Ongoing: Rise of higher level tools on top of Hadoop, such as Pig, Hive, Sqoop, Flume, and Oozie, followed by newer engines such as Spark, and fully managed cloud platforms such as

Google BigQuery.

CHECK YOUR UNDERSTANDING (Section 6)

1. Describe the historical evolution of Big Data technologies, and state the key milestones.
(Question Bank Q6)
2. What is meant by the “Data Deluge,” and what specific numbers illustrate it?
3. Name the three phases of Big Data evolution and the dominant data type in each phase.

SECTION 7: GOOGLE FILE SYSTEM (GFS)

7.1 Origin and Purpose

Developed by Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, introduced in the 2003 paper “The Google File System.” It was designed to meet Google’s rapidly growing data processing needs, providing a scalable, fault tolerant distributed file system built on inexpensive, commodity hardware. Used internally to support applications such as Google Search, Google Earth, and Gmail.

7.2 Why Not Use Existing File Systems

NFS (Network File System) design characteristics:

- Originally designed for managing a large number of small files, typically using small block sizes (8KB to 16KB), resulting in relatively high metadata overhead.
- Primarily supports scaling up (growing the volume), and lacks native support for scaling out.
- As workloads grew beyond what a single machine could handle, the need for distributed operations became clear. NFS was not designed for this, and solutions like Clustered Shared Volumes in NTFS were retrofitted later.

HDFS was built specifically to scale out across clusters, based on the open source implementation of GFS, designed from the ground up for distributed, fault tolerant storage.

7.3 GFS Architecture

A GFS cluster has a simple, centralized architecture consisting of a single Master node and multiple Chunk Servers.

- Master Node: stores all file system metadata, including the namespace, access control information, and the mapping of files to chunks.
- Chunk Servers: store the actual data chunks on local disks.
- Chunk Size: GFS uses a large, fixed chunk size of 64 MB. This reduces the amount of metadata the master needs to manage and minimizes client master interaction for sequential reads and writes.
- Replication: each chunk is replicated across multiple chunk servers (default of three) to provide fault tolerance and increase read throughput.

7.4 GFS Operations

- Read and Write Operations: clients query the master for chunk locations, then communicate directly with the chunk servers.
- Atomic Record Appends: designed for concurrent writes where multiple clients can append to the same file simultaneously.
- Snapshots: efficiently copy data for backup or to create replicas.

7.5 Fault Tolerance in GFS

- Heartbeat Messages: the master sends regular heartbeat messages to chunk servers to ensure they are alive.
- Chunk Replication: ensures that data is replicated and available even if some chunk servers fail.
- Data Integrity: each chunk server uses checksums to detect data corruption. Before returning any data, it verifies the checksum of the blocks. If a mismatch is found, it reports an error and the master arranges to clone a new, valid replica.

7.6 Performance and Scalability

- Caching: clients cache metadata to reduce master load.
- Load Balancing: distributes load evenly across chunk servers.
- Recovery: the master quickly re-replicates chunks when chunk servers fail, to maintain redundancy.
- Master Node Bottleneck: the system is designed to minimize the load on the master by keeping operations involving the master to a minimum.
- Adding Chunk Servers: easily scalable by adding more chunk servers.

7.7 Diagnostic Tools

Extensive and detailed diagnostic logging helps in problem isolation, debugging, and performance analysis at minimal cost. Logs record significant events, such as chunk servers going offline or coming back online, and every RPC (Remote Procedure Call) request and its corresponding reply, giving a complete chronological record of client server and master chunkserver interactions. These logs can be freely deleted without affecting correctness, but are kept as long as space permits.

7.8 Strengths and Limitations of GFS

Strengths:

- Handles very large files efficiently across commodity hardware.
- Strong fault tolerance through replication and checksums.

- High throughput for large, sequential reads and writes, well suited to web scale batch processing.
- Simple, centralized metadata management via a single master keeps the design conceptually simple and consistent.

Limitations:

- The single Master node is a potential bottleneck and a single point of failure for metadata operations, even though the system tries to minimize master involvement.
- Large 64 MB chunk size is inefficient for workloads involving many small files.
- Optimized for write once, read many access patterns, and is not well suited to low latency or frequent small random writes.
- Not designed for real time or interactive workloads, since its strength is high throughput batch style access.

CHECK YOUR UNDERSTANDING (Section 7)

1. Explain the working and design of the Google File System. How is it similar to HDFS?
(Question Bank Q22)
2. What are the strengths and limitations of GFS in handling large scale web data? (Question Bank Q23)
3. Why does GFS use a large, fixed chunk size of 64 MB instead of small blocks like traditional file systems?
4. Describe the role of checksums in maintaining data integrity in GFS.
5. Why is the GFS Master node considered a potential bottleneck, and what design choices are made to reduce this risk?

SECTION 8: HDFS (HADOOP DISTRIBUTED FILE SYSTEM)

8.1 Why HDFS

Example: 1.3 billion credit or debit card transactions yield 13 TB of data. This massive data needs efficient storage and processing, which HDFS is designed to handle.

8.2 Introduction

- Stores very large files efficiently.
- Operates on clusters of commodity hardware.
- Designed for write once, read many times access.
- Optimized for high throughput data processing on Big Data, not low latency.

8.3 File System Fundamentals

- Disks are block oriented devices, meaning data transfer occurs in blocks, for example 4KB.
- A file system provides a structured way to store and retrieve data, so that information is separated and grouped into named files rather than one undifferentiated body of data.
- Desirable properties of files: long term existence, sharable between processes (with access permissions), and structure (hierarchical organization reflecting relationships among files).
- File system design principles:
 - Separation of Data and Metadata: Metadata is information about the file (name, size, permissions, location on disk), while Data is the actual content.
 - Block Mapping: file systems manage the mapping between logical files and the physical blocks on disk.
 - Different file systems, such as NTFS and FAT32, have distinct structures, performance characteristics, and limitations (for example, FAT32 has a 4GB file size limit), impacting speed, flexibility, security, and maximum storage size.

8.4 Local File Systems versus Distributed File Systems

Local File System: forms an intermediate layer between applications and block oriented disks, along with layers of volume management software (RAID controllers, virtualization systems). Generic, supports all kinds of applications, and is not tuned for the performance of any particular application type.

Distributed File System (DFS): used when data is too large to fit on a single disk.

- Manages files and folders across multiple computers.
- Organizes and displays files as if they are stored on one computer, serving the same purpose as a traditional file system.
- Designed to provide file storage and controlled access to files over local and wide area networks.

8.5 HDFS Definition

“HDFS is a filesystem designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.”

- Very large: files can be MB, GB, or TB in size, and Hadoop clusters at the PB scale are currently operational.
- Read mostly data: the most efficient data processing pattern is write once, read many times. Each analysis involves a large proportion of the dataset, so the time to read the whole dataset matters more than the latency in reading the first record.
- Commodity hardware: Hadoop does not require expensive, highly reliable hardware, and is designed to run on clusters of commodity hardware.

8.6 Comparison of HDFS with Traditional File Systems

Aspect	Traditional File System	HDFS
Scale	Single machine or small server, limited by local disk size	Distributed across a cluster, scales by adding nodes
File size focus	Optimized for many small to medium sized files	Optimized for very large files (block size 128 MB)
Access pattern	Supports arbitrary read and write at any offset	Write once, read many, append only at end of file
Fault tolerance	Typically relies on RAID or backups at hardware level	Built in replication across multiple nodes (software level)
Latency	Low latency, suited to interactive access	High throughput focus, not optimized for low latency

Aspect	Traditional File System	HDFS
Metadata handling	Stored alongside data on the same disk	Centralized in NameNode, kept separately from DataNodes
Compliance	Usually POSIX compliant	Not POSIX compliant

8.7 HDFS Master Slave Architecture

Metadata versus Data (the motivation for the split):

	File Metadata	File Data
Content	Filename, access control, size, location	The actual data of the file
Size	Small	Much larger
Access frequency	Accessed less frequently	Most access time is spent here

Solution: keep metadata on a separate server (NameNode), since metadata is smaller and accessed less often, and distribute data across many machines (DataNodes), since data is much larger and requires parallel access.

8.8 NameNode

- The Master Daemon.
- Maintains and manages DataNodes.
- Stores information about block locations, permissions, and so on, on the local disk in two files:
 - Fsimage (File System image): contains the complete namespace of the Hadoop file system since NameNode creation.
 - Edit log: contains all recent changes performed to the file system namespace since the most recent Fsimage.
- Receives heartbeat and block reports from all the DataNodes.

8.9 DataNodes

- The slave nodes and the workhorses of HDFS.
- Inexpensive commodity hardware storing blocks of a file.

- Responsible for serving client read and write requests.
- Based on instructions from the NameNode, perform block creation, replication, and deletion.
- Send a heartbeat to the NameNode to report the health of HDFS.
- Also send block reports to the NameNode listing the blocks they contain.

8.10 Interaction Between NameNode and DataNode

- DataNodes periodically send heartbeat signals to the NameNode to confirm they are alive and functioning.
- DataNodes periodically send block reports listing all the blocks they currently hold, allowing the NameNode to maintain an accurate map of block locations.
- When a client wants to read or write a file, it first contacts the NameNode to get the metadata (which blocks make up the file, and on which DataNodes those blocks live). The client then communicates directly with the relevant DataNodes for the actual data transfer, the NameNode itself does not move data.
- The NameNode instructs DataNodes to create, replicate, or delete blocks as needed, for example to maintain the configured replication factor.

8.11 Secondary NameNode

- A helper node to the primary NameNode.
- Provides housekeeping backup, but is not a hot standby.
- Supports the primary NameNode by downloading and merging the Fsimage file and edit logs file, updating the Fsimage and refreshing the edit logs.
- Sends this updated image back to the NameNode, enabling the NameNode to start faster after a restart.

8.12 HDFS Blocks

- Disk blocks: minimum data that can be read or written, typically 512 bytes.
- HDFS blocks: a much larger unit, 128 MB in Hadoop v2 and v3 (default).
- Files in HDFS are broken into block sized chunks, stored as independent units.
- A file smaller than a single block does not occupy a full block's worth of underlying storage, it uses only as many disk blocks as necessary.

Benefits of block abstraction:

- A file can be larger than any single disk in the network.

- Files can be distributed across disks.
- Simplifies the storage subsystem.
- Blocks fit well with replication for providing fault tolerance and availability.

Useful commands:

- `hadoop fsck -files -blocks` lists the blocks that make up each file in the filesystem.
- HDFS fsck is used to check the health of the file system, finding missing files, over replicated, under replicated, and corrupted blocks.

8.13 Basic HDFS Operations

Command	Purpose
<code>hdfs dfs -ls /</code>	List the files and directories in the root directory
<code>hdfs dfs -mkdir /mydir</code>	Create a new directory named mydir
<code>hdfs dfs -put localfile /mydir/</code>	Copy localfile from the local file system into HDFS
<code>hdfs dfs -get /mydir/remotefile .</code>	Copy remotefile from HDFS to the current local directory
<code>hdfs dfs -rm /mydir/remotefile</code>	Delete a file from HDFS

8.14 Fault Tolerance in HDFS

Fault tolerance in HDFS refers to the working strength of the system under unfavorable conditions and how it handles such situations. HDFS is highly fault tolerant.

What happens when a DataNode fails:

1. The failed DataNode stops sending heartbeat signals to the NameNode.
2. If the NameNode does not receive a heartbeat from a DataNode within a set time window, it marks that DataNode as dead or unreachable.
3. The NameNode identifies which blocks were stored on the failed DataNode (using its block reports history).

4. Since each block is replicated across multiple DataNodes (by default, per the replication factor), the data remains accessible from the surviving replicas on other machines.
5. The NameNode instructs other, healthy DataNodes to create new replicas of the affected blocks, to restore the configured replication factor and maintain fault tolerance for future failures.
6. Tasks or jobs relying on data from the failed node are transparently redirected to read from a surviving replica.

Two main approaches to fault tolerance in HDFS:

a) Replication (up to Hadoop 3):

- Fault tolerance of data is achieved by creating replicas of user data based on the replication factor, on different machines in the HDFS cluster.
- If any machine in the cluster goes down, data remains accessible from other machines that hold the same copy of data.

b) Erasure Coding (introduced in Hadoop 3, as an alternative to replication):

- Works similar to RAID, striping the file into small units of sequential blocks and storing them consecutively across various disks.
- For each strip of the original dataset, a certain number of parity cells are calculated using an erasure coding algorithm called encoding, and stored.
- If any machine fails, the block can be recovered from the parity cells, a process known as decoding.
- Erasure coding reduces the storage overhead to 50 percent, compared to the higher overhead of full replication.

8.15 HDFS Failover and High Availability

Failover Controller:

- Handles the transition from active to standby NameNode.
- Failover controllers are pluggable, using ZooKeeper to ensure that only one NameNode is active at a time.
- Can be manually triggered for maintenance, or triggered automatically.
- A concern is distinguishing an ungraceful failure (a real crash) from a false alarm (for example, a slow network making an active node appear failed when it is actually still running).

HDFS uses ZooKeeper for:

- Failure Detection: the NameNode maintains a pe

rsistent session with ZooKeeper. If the machine crashes, the session expires, and ZooKeeper notifies the other NameNode of the crash.

- Active NameNode Election: on a crash, the other node takes an exclusive lock, indicating that it is the new leader.

HDFS High Availability (post Hadoop 2.x):

- Objective: make the NameNode resilient to failure, and ensure it can recover quickly.
- Configuration: an Active, Standby setup, where edit logs are stored on shared storage, and block mappings are stored in memory.
- On Failure: the Standby NameNode takes over, using the shared edit logs and its in-memory block mappings.
- This provides transparency to clients and a reliable failover mechanism.

8.16 Where HDFS Does Not Score Well

- Low latency data access: applications requiring low latency access to data, in the tens of milliseconds range, are not well served, since HDFS throughput can come at the cost of latency.
- Lots of small files: since the NameNode holds all filesystem metadata in memory, the number of files in the filesystem is limited by the amount of memory on the NameNode. As a rule of thumb, each file, directory, and block takes about 150 bytes, so a million files needs about 300MB of memory, and a billion files would require an impractical amount of memory.
- Multiple writers, arbitrary file modifications: files in HDFS may be written to by only a single writer, and writes are always made at the end of the file. There is no support for multiple writers, or for modifications at arbitrary offsets in the file.
- Not POSIX compliant: applications cannot mount and use standard file system commands with HDFS, and instead need a separate client.

CHECK YOUR UNDERSTANDING (Section 8)

1. Compare and contrast HDFS with traditional file systems, highlighting scalability and fault tolerance. (Question Bank Q8)
2. Illustrate the fault tolerance mechanism in HDFS. What happens when a DataNode fails? (Question Bank Q11)
3. Explain the role of the NameNode and DataNode in HDFS. How do they interact? (Question Bank Q12)

4. Why is the Secondary NameNode not considered a hot standby? What does it actually do?
5. What is the difference between replication and erasure coding as fault tolerance strategies in HDFS, and what is the storage overhead advantage of erasure coding?
6. Name two categories of workloads where HDFS performs poorly, and explain why in each case.

SECTION 9: COMPUTATION MODEL

9.1 Background

Data is stored in a distributed fashion using HDFS. The next requirement is to distribute the processing of that data as well.

9.2 Core Idea: Move Compute to Data

The Big Data philosophy is to move compute to data, rather than moving data to compute, which is how traditional centralized systems operate. The programmer writes code as if writing for a single machine, but the framework executes the process in a distributed manner, hiding the messy details:

- Fault tolerance, which provides high availability.
- I/O scheduling.
- Parallelization.

9.3 Motivation for the Model

There is heavy demand for very large scale data processing, leading to the need for scaling.

Scale out, not scale up:

- Symmetric multiprocessing machines (SMP), with a large number of processor sockets (hundreds) and large shared memory (GBs), are significantly more expensive compared to a cluster of low end machines, which are about 4x more cost effective, though many machines are needed.

Building levels of abstraction:

- A simple programming model or interface is used: Map plus Reduce, separating the abstraction of “what” from “how” of data intensive processing.
- An application programmer develops using well defined interfaces, passing on the data (the “what”), and is isolated from system level details such as locking and starvation.
- The framework (MapReduce), designed once and verified for correctness, takes care of the “how.” This includes writing distributed programs with details of threads, processes, and machines, factoring in deadlocks and race conditions.

9.4 Illustrative Example: Distributed Grep

Problem: search for the word “BigData” in a huge text file spread across multiple machines and disks.

```
Split data -> grep -> matches      (in parallel, on each machine)
                |
                cat (combine) -> All matches
```

9.5 Core Insight: Divide and Conquer

- Partition a large problem into smaller sub-problems.
- Workers work on sub-problems in parallel (could be threads in a core, cores in a multi-core processor, multiple processors in a machine, or machines in a cluster), producing intermediate results.
- Intermediate results from workers are combined to form the final result.
- Two basic operations on the input: Map and Reduce.

9.6 Worked Example: Distributed Word Count

Menu 1: Idli, Vada, Pizza. Menu 2: Dosa, Pizza, Burger.

Map phase (parallel): each item is emitted as a (item, 1) pair.

Reduce phase (parallel): counts are aggregated per key.

Key question: how do we know that all the “pizza” keys must be aggregated at the same reducer?
Answer: treat the output of the first program as a key value pair, and use a partitioning function while dividing the keys in the second program, so that all identical keys are sent to the same reducer.

9.7 Formal Computation Model

```
Very big data -> [MAP] -> Partitioning Function -> [REDUCE] -> All matches
```

The partitioning function ensures that all similar keys are aggregated at the same reducer. Each mapper uses the same partitioning function.

- Map: accepts an input key or value pair, and emits intermediate key or value pairs.
- Reduce: accepts an intermediate key with a list of values, and emits an output key or value pair.

9.8 Formal Programming Model (Data Types)

- Data type: key-value records.
- Map function: `(Kin, Vin) -> list(Kinter, Vinter)`
- Reduce function: `(Kinter, list(Vinter)) -> list(Kout, Vout)`

CHECK YOUR UNDERSTANDING (Section 9)

1. Explain the concept of “move compute to data” and why it is preferred over the traditional approach.
2. Why is scaling out with commodity machines generally more cost effective than scaling up with expensive symmetric multiprocessing machines?
3. In the distributed word count example, explain why a partitioning function is essential for correctness.
4. Write out the formal Map function and Reduce function signatures, and explain each part in words.

SECTION 10: MAPREDUCE ARCHITECTURE

10.1 What is MapReduce, and Why Use It

MapReduce is the central processing component of the Hadoop framework. It provides a programming model for processing vast amounts of data in a distributed environment.

- Origin: from Google, published at OSDI 2004, built on principles of parallel and distributed computing.
- Hadoop is the open source adoption of this idea by Yahoo, now an Apache project.
- Key advantages:
 - Parallel Processing: enables the parallel processing of data across multiple servers in a distributed computational environment, significantly boosting performance.
 - Data Locality: a fundamental principle, where instead of moving large amounts of data to the processing unit, the framework moves the processing logic to where the data is already stored, reducing expensive network traffic and improving efficiency.
- It is an execution framework for large scale data processing:
 - Distributes computing across distributed commodity servers, then pulls the results together.
 - Hides messy details such as fault tolerance, I/O scheduling, and parallelization.

10.2 The Two Main Phases of MapReduce

Phase 1: Map

- Processes an input key or value pair to generate intermediate key or value pairs.
- Sorts all key or value pairs before sending them to the reducer.

Phase 2: Reduce

- Merges all intermediate values associated with the same key.
- Runs only after all Map tasks are finished, because a given key's data could originate from any or all of the mappers.

Between the two phases sits the shuffle and sort step, which transfers and groups Map output by key before it reaches the Reducer.

10.3 Map Reduce Word Count Example (Python Logic)

Mapper logic:

- Read each line from input.
- Strip whitespace, split the line into words.
- For each word, print `word (tab) 1` (a tab delimited key-value pair).

Reducer logic:

- Read each (key, value) line, split by tab.
- Skip invalid, non-integer values.
- If the current key equals the previous key, accumulate the running sum.
- Otherwise, print the previous key's total, then start a new running sum for the new key.
- At the end, print the last key's total.

10.4 MapReduce Example: Search

- Input: `( lineNumber, line )` records, and a search pattern.
- Output: line numbers where the pattern occurs.
- Map: if a line matches the pattern, output the line number.
- Reduce: identity function. It is possible to skip the reducer entirely, running this as a map only job.

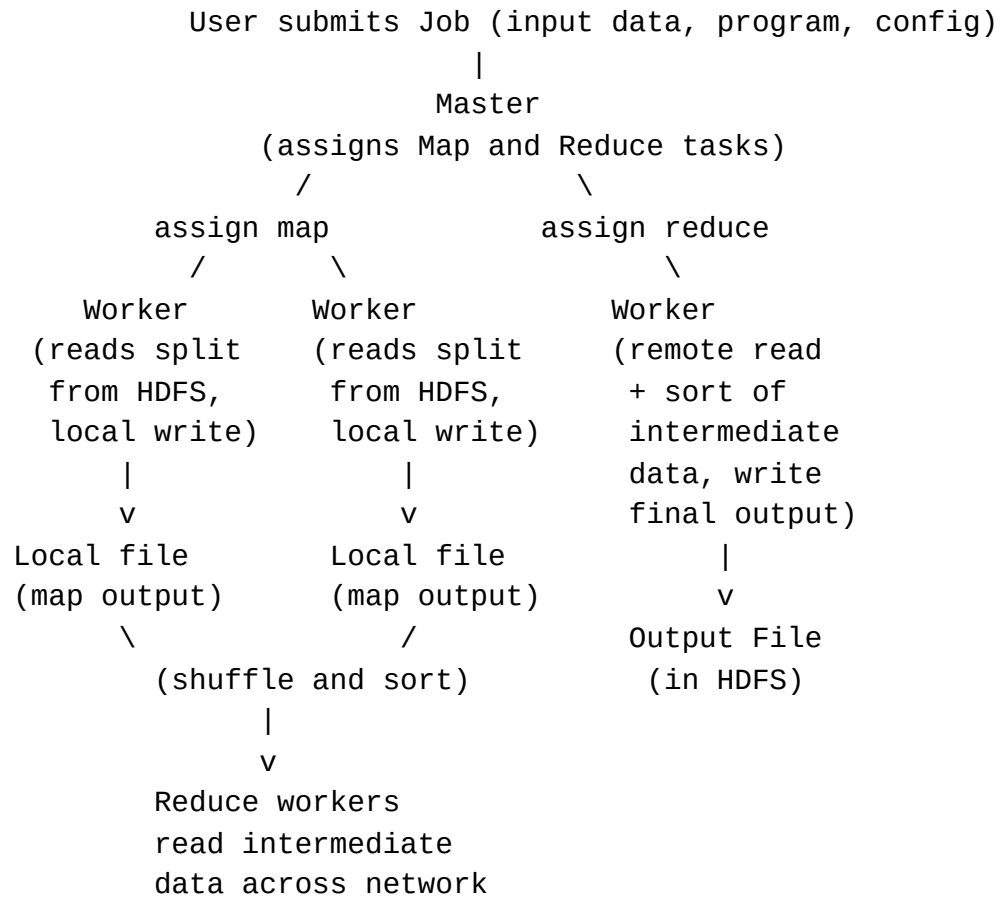
10.5 MapReduce Example: Sort

- Input: (key, value) records. Output: the same records, sorted by key.
- Map: identity function. Reduce: identity function.
- Trick: choose the partitioning function p such that if k_1 is less than k_2 , then $p(k_1)$ is less than $p(k_2)$. This works because Map's output is already sorted by key.

10.6 Job Submission Flow

1. The user submits a job: input data, the MapReduce program, and configuration information.
2. Parallelization is achieved by:
 - Dividing the input into smaller chunks, called input splits.
 - Hadoop divides jobs into tasks: Map tasks and Reduce tasks.
 - One map task runs per split, and tasks run in parallel.

10.7 Block Diagram of MapReduce Architecture and Data Flow



Multiple reducer case: outputs are created on two or more different nodes, and must eventually be merged, but they remain available through HDFS from any node.

10.8 Split Size Considerations

- Split size is proportional to parallelism.
- Small split size:
 - Advantage: large number of splits, increased parallelism, increased load balancing.
 - Disadvantage: the overhead of managing splits and creating map tasks begins to dominate the total job execution time.
- Optimal split size equals the size of the HDFS block, 128 MB (default) on Hadoop v3.
- Splits are logical partitions of data.

10.9 Map Output and Data Locality

- Split equals block size: all data required for a Map task is on the same node, so no inter-node data transfer is required.

- Split not equal to block size: data transfer across multiple nodes is required, which impacts performance.

10.10 Where Map Output is Stored

- Written to local disk, not HDFS.
- Reason: it is a temporary output, discarded after being consumed by the reducer.
- Failure handling: if the node running the map task fails before the output has been consumed by the reducer, the map task is automatically rerun on another node.

10.11 Reduce Tasks

- Reduce tasks do not have the advantage of data locality, since the input to a single reduce task is normally the output from all mappers.
- Sorted map outputs have to be transferred across the network to the node where the reduce task is running.
- Data from different mappers is merged, then passed to the user defined reduce function.
- The output of the reduce phase is normally stored in HDFS for reliability. One replica is written on the local node where the reduce happened, and other replicas are placed on off-rack nodes, which consumes network bandwidth.

10.12 Summary: How MapReduce Achieves Parallelism

1. Parallelize Map: easy, since each map task is independent of the others.
2. Transfer data from Map to Reduce (Shuffle): all Map output records with the same key are assigned to the same Reduce task, using a partitioning function, for example `hash(key) % number of reducers`.
3. Parallelize Reduce: easy, since each reduce task is independent of the others.
4. Storage across the pipeline:
 - Map input: from the distributed file system, HDFS in this case.
 - Map output: to local disk at the Map node, using the local file system.
 - Reduce input: from multiple remote disks, using local file systems.
 - Reduce output: to the distributed file system, HDFS.

10.13 Shuffling: Worked Example (IPL Teams)

Input -> Splitting -> Mapping -> Shuffling -> Reducing -> Output

Splitting divides the input into chunks. Mapping emits a (team, 1) pair for each mention. Shuffling groups all pairs with the same team key together, and routes them to the correct reducer. Reducing sums the counts per team, producing the final output, for example CSK:2, DD:3, KKR:2, RCB:2.

10.14 Combiners

- A Combiner is a “mini reducer” that runs on the Map side, before the data is sent to the actual reducer.
- Purpose: combine multiple map outputs locally, meaning partial aggregation, reducing the amount of data shuffled across the network.
- Written as a function in the program, and it runs before reduce.
- Question: where does a combiner execute? Answer: on the mapper node.

10.15 Worked Review Problems

Problem 1: A 2 GB file, split size 128 MB, and 4 disks (one per node).

- Number of splits: $2048 \text{ MB} / 128 \text{ MB} = 16$.
- Splits per disk: $16 / 4 = 4$.
- Number of map tasks: 16, since there are 16 splits.
- Map tasks per node: $16 / 4 = 4$.
- Number of reduce tasks: user specified.

Problem 2: A 230 MB file, Hadoop v3, block size 128 MB.

- Blocks needed: 2 (one block of 128 MB, one block of the remaining approximately 102 MB), giving 2 mappers.
- Number of reducers: configurable, set by the user.

Problem 3: Record spanning a block boundary.

- Each split is further broken into records. A mapper near a block boundary needs to handle incomplete records by looking for an End of Record (EOR) separator, which is `\n` by default, and can be changed. A mapper reads slightly past its own split boundary to complete its last record, and skips a partial first record it does not own, so the final word count result is still correct.

CHECK YOUR UNDERSTANDING (Section 10)

1. Explain the working of the MapReduce computational model, and state the two main phases.
(Question Bank Q7)
2. How does MapReduce achieve parallelism in large scale data processing tasks? (Question Bank Q9)
3. With a block diagram, explain the MapReduce architecture and data flow. (Question Bank Q10)
4. Why is Map output stored on local disk instead of HDFS, and what happens if the node running a Map task fails before the output is consumed?
5. Explain, with the IPL example or your own example, how shuffling groups keys before the reduce phase runs.
6. Solve: a 1 GB file with a split size of 128 MB is processed on a 2 node cluster. How many splits, how many map tasks, and how many map tasks per node (assume even distribution)?

SECTION 11: HADOOP ECOSYSTEM

11.1 Overview

HDFS provides storage and MapReduce provides computation. Around these two core components sits a wider ecosystem of tools, each solving a specific problem in the Big Data pipeline.

Tool	Purpose
Pig	A scripting language that simplifies data analysis, avoiding complex low level MapReduce code
Hive	Enables SQL like queries (HiveQL) to analyze data stored in Hadoop
Sqoop	Efficiently transfers data between structured relational databases and Hadoop
Flume	Ingests and moves large streams of log or event data into Hadoop
Oozie	Schedules and manages complex, interdependent, multi-step workflows
HBase	A NoSQL database providing fast, real time read and write access on massive datasets

11.2 Pig (Brief)

Apache Pig is a high level language that lets programmers write complex MapReduce transformations using a simple scripting language called Pig Latin. It was introduced by Yahoo as an open source, high level data flow system.

Why Pig, versus writing MapReduce directly:

- Manual coding of common operations like JOIN and FILTER in raw MapReduce is tedious, hard to read, and error prone.
- Pig provides built-in support for joins, filters, ordering, sorting, and nested data types (tuples, bags, maps).
- Pig Latin scripts are internally converted to MapReduce jobs, but require fewer lines of code and less development effort than hand written MapReduce, at a small cost to raw code efficiency.

CHECK YOUR UNDERSTANDING (Section 11)

1. Describe the components of the Hadoop Ecosystem, and highlight their individual roles.
(Question Bank Q13)
2. Why does raw MapReduce programming create a need for tools like Pig?
3. What kind of data operations does Pig make significantly easier compared to writing MapReduce code directly?

SECTION 12: APACHE HIVE AND HIVEQL

12.1 What is Hive

Hive is the open source data warehouse infrastructure system used to process structured data in Hadoop. It is similar to SQL, providing an easier way to query, analyze, summarize, and manage structured data in HDFS, while being built on top of MapReduce and Hadoop.

It was initially developed by Facebook's infrastructure team to support:

- Structured logs with rich data types (structs, lists, and maps).
- A user base wanting to access this data in the language of their choice.
- Traditional SQL workloads on this data, such as filters, joins, and aggregations.

12.2 How Hive Works

- Foundation: Hive operates on data that is already stored in HDFS.
- HiveQL: users write queries in HiveQL, a language very similar to standard SQL.
- Metastore: a central repository storing the schema, or metadata, for the data in HDFS, which allows Hive to interpret the data as structured tables.
- The Engine: Hive translates a HiveQL query into one or more MapReduce jobs, which are then executed on the Hadoop cluster to produce the result.

12.3 Case Study: How Hive Simplifies Querying Large Datasets

Instead of writing a custom MapReduce job in Java to filter, join, or aggregate data, an analyst can write a HiveQL query that closely resembles standard SQL. Hive's compiler parses this query, performs semantic analysis, and automatically converts it into a Directed Acyclic Graph (DAG) of MapReduce jobs, coordinating with the Metastore for schema information. This removes the need for Java expertise, letting analysts familiar with SQL work directly on massive, distributed datasets stored in HDFS, without manually managing distributed execution.

12.4 HiveQL versus SQL

HiveQL is very similar to standard SQL, but has some important differences due to the underlying batch, distributed execution model.

Feature	Standard SQL	HiveQL
Update and Delete	Fully supported	Limited or not supported in older versions, since Hive was not designed for row level transactions
Execution	Often near instantaneous	Query is translated into MapReduce jobs, so there is higher latency
Real time queries	Yes, typically low latency	No, Hive queries have high latency, unsuited to real time use
Schema	Schema on write, enforced at insert time	Often schema on read, enforced at query time
Underlying storage	Native database engine, proprietary storage format	Files stored in HDFS, in formats like text, ORC, or Parquet

Example query comparison:

Standard SQL:

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000
GROUP BY department;
```

HiveQL:

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 50000
GROUP BY department;
```

The syntax itself is nearly identical in this simple example. The key difference is not the syntax but the execution: the SQL version runs directly against a database engine, while the HiveQL version is compiled into a MapReduce (or similar) job that runs across the Hadoop cluster, with much higher latency but far greater scalability for large datasets.

12.5 Advantages and Limitations of Hive

Advantages:

- Removes the need for expertise in Java or a programming language to use Hadoop MapReduce.
- Provides SQL like queries, HiveQL, that are implicitly transformed into MapReduce or Spark jobs.
- Capable of analyzing large datasets stored in HDFS.
- Allows different storage types such as plain text, RCFile (Record Columnar File), and HBase.
- Can operate on compressed data stored in the Hadoop ecosystem.
- Supports user defined functions (UDFs), where a user can provide their own custom functionality.

Limitations:

- Not capable of handling real time data.
- Not a full database. Hive does not fully support update, alter, and deletion of records.
- Not designed for online transaction processing.
- Not developed for unstructured data.
- Hive queries have high latency.

12.6 Hive Data Model: Tables, Partitions, and Buckets

Hive data is typically stored as HDFS files and can be categorized into three granular levels:

Table:

- Similar to a table in a relational database.
- Mapped to an HDFS directory.
- Operations like filter, project, join, and union can be performed on tables.
- For scalability, tables are divided into multiple files and directories.

Partition:

- Tables are organized into partitions based on a column, called the partition key, grouping similar data together.
- Implemented as an HDFS subdirectory.
- Example: a table of sales records could be partitioned by date, so each date's data lives in its own subdirectory. A query filtering on a specific date only needs to scan that one partition's

directory, instead of the entire table.

Bucket:

- A subdivision of a partition.
- Divided based on the hash function of a specified column, to enhance query efficiency.
- Example: within a single date partition, records could be bucketed by user ID hash, allowing more efficient sampling and joins.

How partitions and buckets enhance performance:

- Partitioning reduces the amount of data scanned for a query, since only the relevant partition directories need to be read, rather than the entire table. This is especially useful when queries commonly filter on the partition column, such as date or region.
- Bucketing further subdivides data within a partition using a hash function, enabling more efficient joins between two bucketed tables (since matching hash buckets can be joined directly) and enabling efficient sampling of a subset of data without scanning everything.

12.7 Hive Components

- WebUI or Hive CLI: WebUI is a web based GUI, and Hive CLI (Command Line Interface) is a shell for executing Hive queries and commands.
- Thrift Server: a cross language service provider platform that serves requests from any programming language that supports Thrift.
- JDBC Driver: establishes a connection between Hive and Java applications.
- ODBC Driver: allows applications supporting the ODBC protocol to connect to Hive.
- Hive Driver: receives queries from sources like the web UI, CLI, Thrift, and JDBC/ODBC drivers, and transfers the queries to the compiler.
- Hive Compiler: parses the query, performs semantic analysis on query blocks and expressions, creates the execution plan while communicating with the Metastore for metadata, and converts HiveQL statements into a DAG of MapReduce jobs.
- Hive Execution Engine: executes the incoming tasks in the order of their dependencies.
- Hive MetaStore: a central repository storing all structural information of tables and partitions, including column and type information, and the location of the corresponding HDFS files.
- Hive Server: also known as the Apache Thrift Server, accepts requests from different clients and provides them to the Hive Driver.

- SerDe (Serializer/Deserializer): reads data from disk and converts it to Hive's internal format, and does the reverse during writes, supporting formats like Thrift and JSON.

CHECK YOUR UNDERSTANDING (Section 12)

1. Case Study: describe how Hive simplifies querying and analyzing large datasets. What is its query language? (Question Bank Q19)
2. How does HiveQL compare with SQL? Give examples of queries in both. (Question Bank Q20)
3. What are partitions and buckets in Hive? How do they enhance performance? (Question Bank Q21)
4. Name two limitations of Hive that would make it unsuitable for a real time transaction processing system.
5. Explain the role of the Hive Metastore in translating a HiveQL query into an executable job.

SECTION 13: APACHE FLUME AND KAFKA

13.1 Context of Flume

Big Data comes from various data sources, such as application servers, social networking sites, cloud servers, and enterprise servers, often in the form of log files and events. Getting this data into HDFS using simple commands like “put” requires the data to be packaged as files and transferred one file at a time. In scenarios like web servers, where data is continuously generated, this leads to challenges of delay and reliability. Flume is designed to solve this.

13.2 What is Apache Flume

Apache Flume is a tool, service, or data ingestion mechanism for effectively collecting, aggregating, and moving large amounts of data. It is based on streaming data flows, collecting large amounts of streaming data into a centralized store, from sources such as Events and Logs (like web server logs). It is distributed, reliable, and fault tolerant, with failover and recovery mechanisms.

13.3 Flume Architecture

- **Agent:** receives Flume events from data generators, and stores them into a centralized store like HDFS or HBase. Agents are independent daemon processes (JVM instances).
- **Source:** part of an Agent, receives data from the data generators, and transfers it to one or more channels in the form of Flume Events.
- **Flume Event:** the basic unit of data transport inside Flume, containing a payload of a byte array to be transported from source to destination.
- **Sink:** receives data and stores it into centralized stores like HDFS or HBase. It consumes the data from the channels and delivers it to the destination.
- **Channel:** connects sources to sinks. It is a transient store that receives events from the source and buffers them until they are consumed by sinks, acting as a bridge between the source and the sink, for example a JDBC channel or a file system channel.

13.4 Flume as a Data Ingestion Tool for Streaming Logs

Flume acts as an ingestion pipeline specifically for continuous, streaming log data. A Source (for example a web server’s log tailer) continuously reads new log lines as they are written. These become Flume Events, which flow into a Channel, acting as a buffer so that a temporary slowdown at the destination does not lose data. A Sink continuously drains the Channel, writing the events into

HDFS or HBase in near real time. This design allows continuous, streaming ingestion, rather than the batch, one file at a time approach of manual HDFS commands.

13.5 Apache Kafka (General Comparison Context)

Apache Kafka is a distributed event streaming platform, commonly used for building real time data pipelines and streaming applications. While it is not part of the classic slide deck content, it is commonly compared with Flume because both are used to move streaming data.

13.6 Flume versus Kafka

Aspect	Apache Flume	Apache Kafka
Primary purpose	Purpose built data ingestion tool, mainly for moving log and event data into Hadoop related stores (HDFS, HBase)	General purpose distributed messaging and event streaming platform, usable for many downstream consumers
Architecture	Agent based, with Source, Channel, and Sink components, typically a pipeline oriented design	Publish subscribe based, with Producers, Topics (partitioned logs), and Consumers
Data retention	Data is typically not retained long term inside Flume itself, it flows through and is delivered to a sink	Kafka retains messages in topics for a configurable period, allowing multiple consumers to read the same data independently, even after it was produced
Consumers	Usually tied to a single destination sink per flow (though fan out is possible)	Naturally supports multiple, independent consumer groups reading the same topic
Coupling to Hadoop	Closely associated with the Hadoop ecosystem, designed with HDFS or HBase sinks in mind	General purpose, not tied to Hadoop, commonly used as the backbone for many different types of real time systems
Typical use case	Ingesting server logs into HDFS for later batch analysis	Building real time pipelines, powering multiple downstream applications (analytics, monitoring, alerting) simultaneously from the same stream

Aspect	Apache Flume	Apache Kafka
Fault tolerance model	Reliable, with failover and recovery mechanisms built around agents and channels	Distributed, replicated log with strong durability guarantees across a broker cluster

In short, Flume is best understood as a specialized, Hadoop oriented log collection pipeline, while Kafka is a more general purpose, durable, replayable streaming backbone that supports many independent consumers.

CHECK YOUR UNDERSTANDING (Section 13)

1. What is Apache Flume? How does it act as a data ingestion tool for streaming logs? (Question Bank Q14)
2. Differentiate between Apache Flume and Apache Kafka in terms of architecture and use cases. (Question Bank Q15)
3. Name the four core architectural components of Flume, and describe the role of each.
4. Why can Kafka support multiple independent consumers reading the same data, while Flume typically cannot, in its basic design?

SECTION 14: YARN (YET ANOTHER RESOURCE NEGOTIATOR)

14.1 Motivation

The previous sections cover distributed data processing (MapReduce) and job submission flow. The remaining question is how job management actually happens, meaning how resources are allocated, how tasks are monitored, and how failures are handled. This is addressed by YARN.

14.2 Terminology

- Job: the entire MapReduce application.
- Task: an individual mapper or reducer within a job.

Core questions YARN answers:

- How to allocate resources, meaning which nodes will run which jobs.
- How to monitor tasks, meaning starting new tasks or restarting failed or slow tasks.
- How to monitor the overall state of the job.

14.3 Hadoop 1.0 Job Management (the older design)

- JobTracker: managed cluster resources and job scheduling, on a single machine.
- TaskTracker: one per task, managing that individual task.
- Fault tolerance, cluster resource management, and scheduling were all handled by the JobTracker.

14.4 Hadoop 1.0 Issues (which led to the creation of YARN)

1. Limits scalability: the JobTracker ran on a single machine and was responsible for cluster management, scheduling, and monitoring, all at once.
2. Availability: the JobTracker was a single point of failure.
3. Resource utilization problems: predefined map and reduce slots caused wasted capacity, for example if map slots were full but reduce slots were free.
4. Limitation in running applications: tightly integrated with Hadoop, so only MapReduce applications could run, and could not coexist with other application types.

14.5 YARN Architecture

YARN was introduced in 2010 with MapReduce version 2, needed to manage clusters exceeding 4000 nodes. It splits the responsibility that JobTracker used to hold, into two separate components:

- Resource Manager: manages cluster wide resources.
- Application Master: manages the lifecycle of a single application.

14.6 Core Components of YARN

Component	Functionality
Resource Manager	Arbitrates resources amongst all applications of the system
Node Manager	A per machine slave, responsible for launching application containers and monitoring resource usage
Application Master	Negotiates appropriate resource containers from the scheduler, and tracks and monitors the progress of the containers
Container	A unit of allocation incorporating resources such as memory, CPU, and disk

14.7 Execution Flow of a Job Submitted to a YARN Based Cluster

1. The client submits an application (for example a MapReduce job) to the Resource Manager.
2. The Resource Manager's scheduler allocates a Container on some Node Manager to launch the Application Master for this job.
3. The Application Master starts up in that container and registers itself with the Resource Manager.
4. The Application Master negotiates for additional resource Containers from the Resource Manager's scheduler, based on the resource needs of the application (for example, how many map and reduce tasks it needs).
5. Once containers are allocated, the Application Master works with the relevant Node Managers to launch the actual tasks (map or reduce tasks) inside those containers.
6. The Node Managers monitor the resource usage of the containers running on their machine, and report status back.
7. The Application Master continually tracks and monitors the progress of the containers, and can request additional containers or release ones no longer needed.

8. On completion of all tasks, the Application Master informs the Resource Manager that the application has finished, and deregisters itself, releasing its containers.

14.8 Data Locality in MapReduce (under YARN)

Attempts to run the map task on a node where the input data resides in HDFS, since this data locality optimization avoids using valuable cluster bandwidth. If all nodes hosting the block replicas are busy, the system looks for a free map slot on a node in the same rack as one of the blocks. Very occasionally, even this is not possible, so an off-rack node is used, resulting in an inter-rack network transfer.

14.9 Schedulers in YARN

Early Hadoop versions used a simplistic FIFO scheduler, meaning jobs run in order of submission, and each job uses the whole cluster while running, resulting in other jobs waiting their turn.

Fair Scheduler:

- Aims to give every user a fair share of the cluster capacity over time.
- Jobs are placed in pools, and by default, each user gets their own pool.
- If a single job is running, it gets all of the cluster. As more jobs are submitted, free task slots are given to jobs in a way that gives each user a fair share, so short jobs complete in a reasonable time, and long jobs still make continuous progress.
- Supports custom pools with guaranteed minimum capacities, defined in terms of map and reduce slots.
- Supports preemption: if a pool has not received its fair share over a certain time, the scheduler will kill tasks in pools running over capacity, to rebalance.

Capacity Scheduler:

- A different approach, using a number of queues, similar to the Fair Scheduler's pools, each with an allocated capacity.
- Queues can be hierarchical.
- Within each queue, jobs are scheduled using FIFO, with priorities.
- Queues cannot use free or spare capacity from other queues, even if it exists, behaving like breaking the cluster up into smaller sub-clusters.

14.10 Handling Failures

Four things that can fail: Task, Application Master, Resource Manager, Node Manager.

Failure	Cause	Detected By	Recovery
Task	Runtime exceptions, hanging tasks, or killed tasks (such as speculative duplicates)	No progress updates for a set timeout, default 10 minutes	Application Master tries restarting the task on a different node
Application Master	Hardware or network failures	Application Master sends periodic heartbeats to the Resource Manager, and a missed heartbeat signals failure	Restarted, up to a maximum number of attempts, default 2
Node Manager	Hardware crash, or slow network	Resource Manager does not receive a heartbeat for 10 minutes	Tasks of incomplete jobs on that node are rerun, possibly on a different node
Resource Manager	More serious, since all tasks fail as a result	Handled by a failover controller	An Active, Standby configuration is used to recover

14.11 Benefits of YARN: Case Study at Yahoo

YARN manages a very large cluster at Yahoo, scaling to over 40,000 servers with 100,000 CPUs and 455 PB of data, running over 850,000 jobs per day. It also provides flexibility, since the same cluster can run Hadoop, Storm, and Spark (on a 100 node cluster) simultaneously, sharing resources using YARN.

14.12 Solved Exercises

- A 1000 node YARN cluster has no jobs running. Two pools are configured with a max of 50 percent of the resources each. A new job requiring 600 nodes is submitted, and on starting, consumes all 600 nodes. Which scheduler is active? Either FIFO or Fair, because both will let a single job use the entire available cluster if there is no other job competing.
- Will the failure of a task result in failure of the entire job? No, the task will simply be restarted.

- What are speculative duplicates? Tasks that are started when the Application Master determines that another running instance of that task is running unusually slowly, as a backup measure.

CHECK YOUR UNDERSTANDING (Section 14)

1. Define YARN. How does it enhance resource management in Hadoop 2.x architecture?
(Question Bank Q16)
2. Describe the execution flow of a job submitted to a YARN based cluster. (Question Bank Q17)
3. What are the core components of YARN, and their respective functionalities? (Question Bank Q18)
4. Why was the JobTracker in Hadoop 1.0 considered both a scalability bottleneck and a single point of failure?
5. Explain the difference between the Fair Scheduler and the Capacity Scheduler, particularly regarding the use of spare capacity from other queues or pools.
6. What happens to a job's tasks if the Node Manager running them fails, and how is this failure detected?

SECTION 15: GOOGLE BIGQUERY

15.1 What is Google BigQuery

BigQuery is described as the autonomous data to AI platform, automating the entire data life cycle, from ingestion to AI driven insights, allowing users to go from data to AI to action faster.

15.2 Key Features

- AI powered conversational and agentic experience.
- A unified data and AI platform for analytics on multimodal data.
- Flexible, low cost ML, with interoperability with open source tools.

Connecting data to AI: Users can create and run ML models using familiar SQL directly within BigQuery, leveraging built in pre-trained models, Vertex AI trained models, or custom models registered in the Vertex AI model registry. This supports multimodal data analysis (document processing, image annotation, audio transcription), generative AI tasks (text summarization, sentiment analysis), and embeddings generation for vector search use cases such as anomaly detection and product recommendations.

Built in AI agents and workflow automation: Gemini in BigQuery provides AI powered experiences and automation. Users can find, join, and query datasets, and visualize results, using natural language prompts in a “data canvas.” It automates data preparation, error detection, and transformations, automatically surfaces useful queries from table metadata, and gives context aware coding assistance. It also gives intelligent recommendations for partitioning, clustering, and materialized views, saving cost and optimizing data infrastructure.

Open source and open formats support: Runs serverless Spark alongside SQL workloads in BigQuery, with unified security, runtime metadata, and governance. BigQuery’s fully managed capabilities, combined with managed Apache Iceberg tables powered by BigLake, enable streaming, advanced analytics, and AI use cases on open formats.

Built in data to AI governance: powered by Dataplex Universal Catalog, providing automatic metadata harvesting, data profiling, data quality checks, and lineage tracking, integrated directly into the BigQuery experience. Gen AI powered capabilities support semantic search, metadata augmentation, and data insights.

Built for enterprise scale and efficiency: BigQuery’s architecture decouples storage and compute for petabyte scale analysis, while optimizing costs with compressed storage, compute autoscaling, and

flexible pricing. Under the hood, BigQuery employs Google infrastructure technologies such as Borg, Colossus, Jupiter, and Dremel. For mission critical workloads, BigQuery also provides managed disaster recovery in the case of a total region outage, relying on cross region dataset replication.

15.3 How BigQuery Supports Real Time Analytics at Scale

BigQuery gains a competitive advantage for real time or event driven analysis. Built in streaming capabilities automatically ingest streaming data and make it immediately available to query, allowing businesses to stay agile and make decisions based on the freshest data. This is supported through:

- Pub/Sub BigQuery subscriptions, which write streamed messages directly to an existing BigQuery table as they are received.
- Dataflow, which enables fast, simplified streaming data pipelines for a more comprehensive real time solution.
- This differs fundamentally from the classic Hadoop MapReduce batch model, since BigQuery's decoupled storage and compute architecture, combined with native streaming ingestion, allows queries to run against continuously updating data, rather than requiring a completed batch job cycle.

15.4 Other Uses of BigQuery

- Migrating data warehouses to BigQuery from systems such as Netezza, Oracle, Redshift, Teradata, Snowflake, or Databricks, using the free BigQuery Migration Service.
- Data integration and ELT (Extract, Load, Transform), the recommended pattern for BigQuery. Batch loading uses BigQuery Data Transfer Service (DTS), streaming load uses Pub/Sub BigQuery subscriptions, and Change Data Capture (CDC) uses Datastream for non-intrusive CDC from databases.
- Simplifying data to AI workflows using Colab Enterprise notebooks, open source Python libraries through BigQuery DataFrames, Jupyter notebooks, and programmatic analysis tools.
- BigQuery data clean rooms, for privacy centric data sharing, allowing collaboration without copying or moving underlying data.
- Geospatial analytics, using rich geospatial datasets, cloud computing, and built in AI tools, without requiring remote sensing or GIS expertise.

15.5 Comparing Google BigQuery with Hadoop Based Data Warehousing Systems

Aspect	Hadoop Based Systems (HDFS, MapReduce, Hive)	Google BigQuery
Infrastructure management	User manages the cluster, nodes, and scaling manually (or via YARN/Ambari)	Fully managed, serverless, no cluster to provision or manage
Storage and compute coupling	Traditionally tightly coupled, storage on DataNodes tied to compute on the same cluster	Decoupled: storage and compute scale independently
Query latency	High latency, since queries are compiled into MapReduce jobs (as with Hive)	Optimized for fast, near interactive SQL queries at scale
Real time support	Weak, MapReduce and classic Hive are batch oriented	Strong, native streaming ingestion supports real time analytics
Scaling model	Scaling requires adding and managing physical or virtual nodes	Autoscaling is automatic and abstracted away from the user
Cost model	Pay for cluster uptime, regardless of whether it is fully utilized	Flexible pricing, often pay per query or per storage, with autoscaling compute
Setup complexity	Significant setup and ongoing administration (Ambari, tuning schedulers, and so on)	Minimal setup, accessible immediately via a managed console or API
Best suited for	Organizations wanting full control over infrastructure, custom tuning, or on premise deployment	Organizations wanting to minimize operational overhead and get fast, scalable analytics with built in AI features

Which scenarios favor each:

- Hadoop based systems are favored when an organization needs full control over its infrastructure, must run on premise for regulatory or cost reasons, or has existing deep investment and expertise in the Hadoop ecosystem.

- Google BigQuery is favored when an organization wants to minimize operational overhead, needs fast interactive or near real time analytics, wants built in AI and governance features without integrating them separately, and is comfortable operating in a cloud, pay per use model.

CHECK YOUR UNDERSTANDING (Section 15)

1. Describe Google BigQuery. How does it support real time analytics at scale? (Question Bank Q24)
2. Compare Google BigQuery with Hadoop based data warehousing systems. Which scenarios favor each? (Question Bank Q25)
3. What role does the separation of storage and compute play in BigQuery's scalability and cost model?
4. Name two BigQuery features that go beyond what classic Hadoop plus Hive can offer out of the box.

SECTION 16: BIG DATA ALGORITHMS: MATRIX MULTIPLICATION

16.1 Matrix Vector Multiplication by MapReduce

Problem: compute $x = M \text{ times } v$, where M is an n by n matrix with entries m_{ij} , and v is a vector of length n with entries v_j .

Result: $x_i = \text{sum over } j \text{ of } (m_{ij} \text{ times } v_j)$

Assumption: v fits in memory, and is available to every Map task. M is huge, and stored in a distributed fashion, row or element wise.

Map function:

- Input: an element m_{ij} of M .
- For each matrix element m_{ij} , produce the pair $(i, m_{ij} \text{ times } v_j)$.
- v is available to all mappers, for example broadcast or loaded into memory ahead of time.

Reduce function:

- For key i , sum all values received: $x_i = \text{sum over } j \text{ of } (m_{ij} \text{ times } v_j)$.
- Output: (i, x_i) .

This follows the same pattern as word count: emit partial products keyed by output row index, then sum them.

16.2 Case Where v Does Not Fit In Memory

If v is too large to fit into a single machine's memory:

- Divide v into horizontal stripes of equal size, and correspondingly divide M into vertical stripes of the same width.
- Each Map task is responsible for one stripe of M , and the corresponding stripe of v , which now fits in memory for that task.
- Each Map task emits partial results $(i, \text{partial value})$ as before, and Reduce sums them exactly as in the simple case.

This ensures each Map task only needs a manageable stripe of v in memory, rather than the whole vector.

16.3 Matrix Matrix Multiplication by MapReduce

Problem: compute $P = M \text{ times } N$, where M is I by J , and N is J by K .

Result: $p_{ik} = \text{sum over } j \text{ of } (m_{ij} \text{ times } n_{jk})$

Approach 1: Single MapReduce Step (One Pass)

Map function:

- For each element m_{ij} of M , for every possible key k , emit $((i,k), ("M", j, m_{ij}))$.
- For each element n_{jk} of N , for every possible key i , emit $((i,k), ("N", j, n_{jk}))$.
- The tags "M" and "N" distinguish which matrix an element came from.

Reduce function:

- For each key (i,k) , group all values by j .
- For matching j , multiply m_{ij} times n_{jk} , and sum over all j : $p_{ik} = \text{sum over } j \text{ of } (m_{ij} \text{ times } n_{jk})$.
- Output: $((i,k), p_{ik})$.

Drawback: this one pass approach can generate a very large intermediate key space (I times K keys, each requiring values from potentially all j), leading to high communication cost.

Approach 2: Two MapReduce Jobs (Two Pass Style)

Job 1: compute all products m_{ij} times n_{jk} (partial products), grouped by j .

- Map 1: emit $(j, ("M", i, m_{ij}))$ for M elements, and $(j, ("N", k, n_{jk}))$ for N elements.
- Reduce 1: for each j , pair up every (i, m_{ij}) with every (k, n_{jk}) , and emit $((i,k), m_{ij} \text{ times } n_{jk})$.

Job 2: sum the partial products by (i,k) .

- Map 2: identity map, passing through $((i,k), \text{value})$.
- Reduce 2: for each key (i,k) , sum all values, giving p_{ik} .

This two stage approach can be more communication efficient for certain matrix shapes and sizes. The right strategy depends on the relative sizes of I , J , and K (see Section 19 for the communication cost model used to compare these strategies).

CHECK YOUR UNDERSTANDING (Section 16)

1. Describe the Map and Reduce functions for matrix vector multiplication when the vector fits entirely in memory.
2. What modification is needed when the vector v does not fit in memory, and why does striping solve this problem?
3. Explain the one pass approach to matrix matrix multiplication, and state its main drawback.
4. Describe the two job approach to matrix matrix multiplication, and explain what each job computes.

SECTION 17: BIG DATA ALGORITHMS: PAGERANK

17.1 Concept

PageRank assigns an importance score to every page on the web, based on the link structure of the web, treated as a directed graph. The core idea: a page is important if important pages link to it.

Model: a “random surfer” starts at a random web page, and repeatedly clicks a random outgoing link. PageRank is defined as the long run probability that the surfer is on a given page.

17.2 The Transition Matrix

Let the web be represented by matrix M , where $M[j][i]$ equals 1 divided by n_i , if page i has a link to page j , where n_i is the number of out-links from page i , and is 0 otherwise. Each column of M sums to 1 (assuming no dead ends), representing an outgoing probability distribution from that page.

17.3 Basic Iterative Formula

$$v_{t+1} = M \cdot v_t$$

v_t is the vector of PageRank scores at iteration t , typically initialized uniformly, $v_0[i]$ equals $1/n$ for all pages. This is repeated until convergence, meaning until the values stabilize or the change falls below a threshold. This is essentially repeated matrix vector multiplication, directly reusing the pattern from Section 16.1.

17.4 Problems With the Basic Model

Dead Ends: pages with no out-links. Their column in M sums to 0, so probability leaks out of the system, and total PageRank shrinks toward 0 across iterations.

Spider Traps: a group of pages that only link to each other, with no links leaving the group. The random surfer gets trapped inside this group forever, so all the importance concentrates unfairly on the trap, while all other pages get PageRank tending to 0.

17.5 The Fix: Teleportation (Random Jumps)

Introduce a damping factor β , typically around 0.85. At each step, with probability β , the surfer follows a random out-link. With probability $(1 - \beta)$, the surfer teleports to a random page anywhere on the web, chosen uniformly.

Modified formula:

$$v_{t+1} = \beta \cdot M \cdot v_t + ((1 - \beta)/n) \cdot e$$

e is a vector of all ones, and n is the total number of pages. This ensures the surfer can always escape dead ends and spider traps, guaranteeing that the process converges to a unique, meaningful stationary distribution.

Dead ends are typically handled additionally by either removing dead end pages recursively before computing PageRank, or by treating a dead end as if it links to all pages, meaning a full teleport from that page.

17.6 PageRank via MapReduce

Each iteration, v_{t+1} equals β times M times v_t , plus the teleportation term, is computed as a matrix vector multiplication (Section 16.1's MapReduce pattern), plus a small constant added to every entry.

Map: for each non-zero $M[j][i]$ and current $v_t[i]$, emit $(j, \beta \text{ times } M[j][i] \text{ times } v_t[i])$.

Reduce: for each destination page j , sum all incoming contributions, then add the teleportation term $(1 - \beta)/n$.

This is repeated for multiple iterations, meaning multiple MapReduce jobs chained together, until convergence.

Note: because v_t must be small enough to fit in memory across mapper tasks, or striped otherwise (per Section 16.2), PageRank computation at web scale needs careful memory and striping management, which ties directly into the communication cost model in Section 19.

CHECK YOUR UNDERSTANDING (Section 17)

1. Explain the random surfer model behind PageRank in your own words.
2. What are dead ends and spider traps, and why do they break the basic PageRank formula?
3. Write out the modified PageRank formula with teleportation, and explain what each term represents.
4. Explain how a single PageRank iteration can be expressed as a MapReduce job.

SECTION 18: ISSUES WITH HADOOP

18.1 Small Files Problem

The HDFS NameNode keeps all metadata in memory, and each file, block, or directory costs approximately 150 bytes. Millions of small files result in hundreds of MB of NameNode memory usage, and billions of files risk exhausting NameNode memory. HDFS is designed for a smaller number of large files, not many small ones.

18.2 High Latency, Batch Oriented Design

MapReduce is optimized for high throughput, batch processing, not for low latency, real time, or interactive queries. It is not suitable for applications needing millisecond level responses.

18.3 Not Ideal for Iterative Algorithms

Many algorithms, such as machine learning and graph algorithms like PageRank, are iterative in nature, meaning each MapReduce job's output must be written to HDFS and re-read from disk for the next iteration. This causes heavy disk I/O overhead across iterations, since MapReduce has no efficient way to keep intermediate data in memory across jobs, unlike newer engines such as Spark, which use in memory data structures.

18.4 Single Point of Failure (Hadoop 1.0)

The JobTracker in Hadoop 1.0 was a single point of failure for job management, and the NameNode, before High Availability was introduced, was a single point of failure for metadata. This was mitigated later by YARN, which splits the JobTracker's role, and by HDFS High Availability, which introduced an Active-Standby NameNode configuration coordinated via ZooKeeper.

18.5 Resource Utilization Issues (Hadoop 1.0)

Fixed map and reduce slots caused wasted capacity, for example map slots being full while reduce slots sit idle, or vice versa. This was fixed later by YARN's flexible, container based resource allocation.

18.6 Limited to the MapReduce Programming Model (Hadoop 1.0)

Original Hadoop was tightly coupled to the MapReduce paradigm only, and could not run other computation frameworks on the same cluster. This was fixed by YARN, which generalized resource management so that Hadoop, Spark, Storm, and other frameworks could coexist on the same cluster.

18.7 Programming Complexity

Writing raw MapReduce jobs in Java for common operations such as JOIN, FILTER, and GROUP BY is low level, verbose, and error prone, which led to the development of higher level abstractions such as Pig and Hive.

18.8 Security

Original Hadoop had minimal built in security. Access control, authentication, and encryption had to be added through external layers, such as Kerberos, which was considered a weakness in early deployments.

18.9 Not POSIX Compliant, Limited File Semantics

HDFS supports only a single writer, with append only writes at the end of a file, with no support for arbitrary offset modification, and no standard filesystem mount or commands.

18.10 Network and Shuffle Overhead

The shuffle phase, meaning the transfer of intermediate Map output across the network to Reducers, can become a major bottleneck for large jobs, which is directly related to the communication cost model discussed in Section 19.

CHECK YOUR UNDERSTANDING (Section 18)

1. List and briefly explain any four major issues with the classic Hadoop 1.0 architecture.
2. Why is MapReduce considered a poor fit for iterative algorithms such as those used in machine learning?
3. How did YARN specifically address the resource utilization and single application type issues of Hadoop 1.0?
4. Explain why the small files problem is a genuine architectural limitation, not just an inconvenience.

SECTION 19: COMPLEXITY OF BIG DATA ALGORITHMS: COMMUNICATION COST COMPLEXITY MODEL

19.1 Why a Special Cost Model for MapReduce

Traditional algorithm analysis uses time complexity, meaning CPU steps, as the primary cost measure. For MapReduce and other distributed algorithms, the dominant cost is usually not CPU computation, but communication, meaning the amount of data that must be written by Map tasks (to local disk) and transferred or read by Reduce tasks (over the network, during shuffle). Hence, MapReduce algorithms are analyzed using a Communication Cost Model, rather than pure computational complexity.

19.2 Definitions in the Communication Cost Model

Communication Cost of a MapReduce Job: the total size of all inputs to all Map tasks, plus the total size of all outputs from all Map tasks that must be transferred to Reducers, meaning the size of the data crossing the Map to Reduce boundary. The cost of a Map task reading its input from distributed storage in isolation is generally not the focus, since the key concern is the key-value pairs produced by Map and consumed by Reduce, as that is the data that must move across the network.

Elapsed Communication Cost: the maximum communication cost of any single task, Map or Reduce, in the job. This models the fact that with enough parallel machines, tasks run simultaneously, so the wall clock bottleneck is the largest single task's cost, not the sum of all tasks.

19.3 Why This Matters: Comparing Algorithm Strategies

The same problem can often be solved by MapReduce in multiple ways, for example matrix multiplication, using the one pass or two pass strategies from Section 16.3. These strategies can have very different communication costs, even if they compute the same correct result. The goal of algorithm design in this model is to minimize communication cost, both total and elapsed, since communication (network I/O) is typically the bottleneck resource in a distributed cluster, not raw computation.

19.4 Applying the Model: Matrix Multiplication Example

One pass approach: for every element of M , a copy is emitted for every possible output key k , and similarly for N , for every possible i . This can create a very large blow up in intermediate data. For example, if M is I by J , and N is J by K , this approach can require communication proportional to I times J times K in the worst case, since each input element is replicated many times.

Two pass approach: intermediate keys are grouped by j first, output pairs are computed via a join, and then summed. Communication cost is typically more balanced and lower, especially when I , J , and K differ significantly in size, since it avoids blindly replicating every element for every possible output key.

Takeaway: the “obvious” algorithm is not always the most communication efficient one. The choice of Map and Reduce strategy directly determines total data shuffled, and hence real world runtime, even for the same underlying computation.

19.5 General Principles for Reducing Communication Cost

1. Minimize data replication: avoid unnecessarily copying or broadcasting large data structures to every Map or Reduce task unless truly required.
2. Use combiners where applicable: partial aggregation on the Map side (Section 10.14) directly reduces the volume of data shuffled to Reducers, lowering communication cost.
3. Choose partitioning and key design carefully: poor key choices can cause data skew, where some reducers get far more data than others, increasing the elapsed communication cost even if the total cost is unchanged, since the job is bottlenecked by the slowest, largest single task.
4. Consider multi-way, one pass strategies versus staged, multi-job strategies: as with matrix multiplication, splitting a computation into more MapReduce stages can sometimes reduce per stage communication volume, at the cost of extra job startup overhead. This is a genuine tradeoff that must be reasoned about explicitly, not assumed.

CHECK YOUR UNDERSTANDING (Section 19)

1. Define Communication Cost and Elapsed Communication Cost, and explain why they differ.
2. Why does the communication cost model matter more than pure CPU time complexity for MapReduce algorithms?
3. Using the matrix multiplication example, explain why the one pass approach can have a much higher communication cost than the two pass approach.
4. What is data skew, and why does it specifically increase elapsed communication cost rather than total communication cost?

QUESTION BANK MAPPED TO SECTIONS (FOR REVISION)

Question No	Topic	Section
Q1	Define Big Data, differ from traditional processing	Section 1
Q2	Five Vs with examples	Section 2
Q3	Challenges in real time Big Data	Section 3
Q4	Opportunities in healthcare, finance, agriculture	Section 4
Q5	Data intensive scientific discovery	Section 5
Q6	Historical evolution and milestones	Section 6
Q7	MapReduce computational model, two phases	Section 10
Q8	HDFS versus traditional file systems	Section 8
Q9	How MapReduce achieves parallelism	Section 10
Q10	MapReduce architecture block diagram	Section 10
Q11	HDFS fault tolerance, DataNode failure	Section 8
Q12	NameNode and DataNode roles and interaction	Section 8
Q13	Hadoop Ecosystem components	Section 11
Q14	Apache Flume for streaming logs	Section 13
Q15	Flume versus Kafka	Section 13
Q16	YARN definition and Hadoop 2.x enhancement	Section 14
Q17	YARN job execution flow	Section 14
Q18	Core components of YARN	Section 14

Question No	Topic	Section
Q19	Hive case study	Section 12
Q20	HiveQL versus SQL	Section 12
Q21	Hive partitions and buckets	Section 12
Q22	Google File System design	Section 7
Q23	GFS strengths and limitations	Section 7
Q24	Google BigQuery, real time analytics	Section 15
Q25	BigQuery versus Hadoop based warehousing	Section 15